Manual Técnico: Contacto 3xtrat3rr3str3D

Autor: Fredy José Gabriel Herrera Funes

Este documento detalla la arquitectura, las herramientas y el flujo interno del compilador **Contacto 3xtrat3rr3str3D**. El compilador procesa tres lenguajes (Pig Latin, Y? y Zetariano) desde un entorno de desarrollo integrado (IDE) propio. El sistema realiza análisis léxico, sintáctico y semántico y genera una representación intermedia en código de tres direcciones (cuartetas). A partir de ella traduce el proyecto a ensamblador RISC-V (main.asm), que se ejecuta en el simulador RARS desde la misma IDE.

1. Tecnologías Utilizadas

El compilador está construido sobre un stack moderno de Java, con herramientas estándar para la generación de analizadores y la gestión del proyecto.

- **Java (OpenJDK):** Versión 21. Lenguaje base. Implementa características modernas como record y switch con patrones.
- **Apache Maven:** Versión 3.8+. Orquestación de construcción, gestión de dependencias y ejecución del proyecto.
- **ANTLR 4:** Versión 4.13.2. Generación de analizadores léxicos y sintácticos a partir de gramáticas .g4.
- **Java Swing:** JDK 21. Desarrollo de la interfaz gráfica de la IDE.
- **RISC-V (RV32IMFD):** Arquitectura destino del código generado. Usa el conjunto base de enteros, multiplicación y división, y punto flotante de precisión simple y doble.
- **RARS:** Versión 1.6. Simulador de RISC-V. Ensambla y ejecuta el archivo main.asm desde la línea de comandos, sin interfaz gráfica.
- **Bash / CMD:** Scripts ejecutar.sh (Linux/macOS) y ejecutar.bat (Windows) que lanzan RARS.
- **draw.io:** Elaboración del diagrama de clases (docs/diagrama_clases.drawio).
- **Git / GitHub:** Control de versiones del código fuente.

Nota de Diseño: No se integran librerías externas para el coloreado de código. El resaltado de sintaxis en la IDE se ejecuta en tiempo real con los lexers autogenerados por ANTLR.

1.1 Configuración de Plugins (Maven)

- **maven-compiler-plugin (3.13.0):** Garantiza la compilación con release 21.
- **antlr4-maven-plugin (4.13.2):** Procesa los archivos .g4 y genera los parsers con la opción visitor activada.

- **exec-maven-plugin (3.5.1):** Permite lanzar la IDE directamente con mvn exec:java.

1.2 Instrucciones de Compilación y Ejecución

Para inicializar el entorno y la IDE desde la terminal:
mvn clean compile
mvn exec:java

Para ejecutar el ensamblador generado, el archivo rars1_6.jar debe estar en la carpeta del proyecto, en su carpeta lib/, o indicado en la variable de entorno RARS_JAR. Se descarga de https://github.com/TheThirdOne/rars/releases.

- **Desde la IDE:** menú Compilar → Traducir y ejecutar.
- **Desde la terminal, en la carpeta del archivo .pig:**

./ejecutar.sh main.asm (Linux / macOS)

ejecutar.bat main.asm (Windows)

Ambos scripts ejecutan internamente java -jar rars1_6.jar nc me main.asm.

2. Distribución y Arquitectura del Proyecto

El código fuente sigue el estándar de Maven y separa las gramáticas de ANTLR de la lógica en Java.

Descripción de Paquetes Core

- **compiler:** Núcleo de orquestación. Detecta el lenguaje por extensión, ejecuta el parser correspondiente, traduce los errores de ANTLR al español y carga la estructura del proyecto resolviendo las importaciones a partir del archivo .pig.
- **compiler.constructores:** Implementa los Visitors de ANTLR que transforman el árbol de derivación de ANTLR en un AST estandarizado.
- **compiler.ast:** Contiene las clases base (Nodo, Tipo, Programa). Define los contratos principales para semántica y traducción.
- **compiler.enviroment:** Maneja el estado de la memoria en tiempo de compilación con TablaSimbolos, ámbitos anidados y registros estructurales.
- **compiler.semantic:** Motor de reglas. Ejecuta las tres pasadas del análisis semántico y evalúa las reglas de TablaCompatibilidad.
- **compiler.c3d:** Orquesta la generación de código. GeneradorC3D y ContextoC3D producen los objetos Cuarteta y usan las rutinas de la clase Nativas. TraductorRiscV convierte las cuartetas en ensamblador RISC-V.

- **gui:** Ventana principal (Ide), editor, árbol de archivos, tabla de errores y consola de ejecución.
- **gui.resaltado:** Motor de análisis léxico en tiempo real para la interfaz gráfica.
- **resources (gui):** Scripts ejecutar.sh y ejecutar.bat, que la IDE copia junto al .asm generado.

3. Flujo Interno del Compilador

La compilación sigue un modelo lineal estricto: un error en cualquier fase detiene el paso a la siguiente.

1. **Carga y Resolución:** Inicia con el archivo .pig principal. Resuelve las instrucciones import a partir de rutas relativas.
2. **Análisis Léxico:** El lexer ignora los comentarios. Un carácter no definido dispara un error léxico.
3. **Análisis Sintáctico:** ANTLR construye el árbol de derivación y aplica estrategias de recuperación de errores.
4. **Construcción del AST:** Los Visitors consolidan los árboles de derivación en una jerarquía común.
5. **Análisis Semántico (3 Pasadas):**
    - **Pasada 1:** Registro de firmas globales.
    - **Pasada 2:** Validación estructural (atributos, firmas, sobrecargas).
    - **Pasada 3:** Verificación profunda de los cuerpos (tipos, ámbitos).
6. **Generación C3D:** Si la tabla de errores está vacía, se recorre el AST para emitir las cuartetas (representación intermedia).
7. **Traducción a RISC-V:** TraductorRiscV convierte cada cuarteta en instrucciones RISC-V y escribe main.asm junto al archivo .pig.
8. **Ejecución:** La IDE lanza el script correspondiente al sistema operativo, que ejecuta main.asm en RARS. La salida se muestra en la pestaña Consola, que también recibe la entrada del usuario.

.pig + .y + .z → Léxico → Sintáctico → AST → Semántico → Cuartetas (C3D) → RISC-V (main.asm) → RARS

4. Análisis Semántico y Sistema de Tipos

El análisis semántico valida la coherencia de tipos, ámbitos y estructuras entre los tres lenguajes (**Pig Latin**, **Y?** y **Zetariano**) mediante el motor del paquete compiler.semantic y el entorno de compiler.enviroment.

4.1 Entorno y Tabla de Símbolos

- **TablaSimbolos y Ámbitos Anidados:** El compilador organiza la memoria en tiempo de compilación mediante ámbitos jerárquicos (global, de clase/estructura, de función/método y bloques locales). Cada declaración registra su identificador, tipo, categoría, visibilidad y desplazamiento relativo (offset) dentro del marco de activación.
- **Resolución de Importaciones:** A partir del archivo principal .pig, el sistema vincula los módulos .y y .z verificando la visibilidad de los símbolos exportados y previniendo colisiones de nombres o dependencias cíclicas.

4.2 Verificación en Tres Pasadas

1. **Pasada 1 (Registro de Firmas Globales):** Recorre las declaraciones de nivel superior en todos los archivos importados para registrar los nombres de estructuras, clases y prototipos de funciones globales, permitiendo referencias adelantadas e interoperabilidad entre lenguajes.
2. **Pasada 2 (Validación Estructural):** Verifica la conformación interna de estructuras (en Pig Latin y Y?) y clases (en Zetariano), validando tipos de atributos, herencia, firmas de métodos, constructores y reglas de sobrecarga sin evaluar aún las instrucciones internas.
3. **Pasada 3 (Verificación Profunda de Cuerpos):** Analiza las sentencias y expresiones dentro de funciones y métodos. Comprueba la existencia y alcance de variables, el uso válido del contexto this, la correspondencia de argumentos en llamadas, la presencia de sentencias return con el tipo esperado y el uso de break / continue dentro de ciclos.

4.3 Sistema de Tipos y TablaCompatibilidad

- **Tipos Primitivos y Compuestos:** Soporta tipos enteros, decimales de doble precisión, booleanos, caracteres, cadenas de texto (string), arreglos multidimensionales, estructuras (struct) y objetos de clases.
- **Reglas de Compatibilidad y Promoción:** La clase TablaCompatibilidad define las matrices de operación para operadores aritméticos, relacionales y lógicos, aplicando promoción implícita (como entero a decimal en operaciones mixtas) y reportando errores semánticos ante asignaciones u operaciones incompatibles.

5. Generación de Código de Tres Direcciones (C3D)

La representación intermedia se expresa como Cuarteta(operador, arg1, arg2, resultado). Es independiente de la arquitectura destino y es la entrada del traductor a RISC-V.

- **Marcos de Función (Stack):** Se reservan posiciones para el valor de retorno, el contexto this, los parámetros y las variables locales.

- **Llamadas y Recursión:** Antes de cada llamada se guardan en el stack todos los temporales activos y después se restauran, simulando el cambio de ámbito.
- **Cadenas (Strings):** Se almacenan en el arreglo global heap como bytes UTF-8 terminados en 0.
- **Arreglos:** Se guardan aplanados en el heap, con una cabecera inicial que indica el tamaño.
- **Estructuras vs Objetos:** En Y? y Pig Latin operan por copia de valor. En Zetariano, los objetos residen en el heap.
- **Control de Flujo:** Se maneja solo con saltos absolutos y condicionales (if ... goto L; / goto L;).
- **Nativas:** Las operaciones sobre cadenas (imprimir, concatenar, convertir entre números y texto, comparar, leer y copiar) están definidas en la clase Nativas, escritas en el mismo código de tres direcciones.

6. Traducción a Ensamblador RISC-V

La clase TraductorRiscV recibe las cuartetas y genera ensamblador RV32IMFD para el simulador RARS.

Modelo de Memoria y Registros

- s0 = P (puntero del marco) y s1 = H (siguiente espacio libre del heap).
- s2, s3 y s4 = direcciones base del stack, del heap y de los temporales.
- Al iniciar, el programa reserva las tres zonas con la llamada sbrk (ecall 9). Cada posición ocupa 8 bytes y guarda un valor double.

Traducción de Cuartetas

- Cada cuarteta carga sus operandos en los registros ft0 y ft1 (con fld, o con li y fcvt.d.w si es un entero literal), opera en doble precisión (fadd.d, fsub.d, fmul.d, fdiv.d, flt.d, feq.d…) y guarda el resultado con fsd.
- El módulo y la conversión a entero usan fcvt.w.d con truncamiento (rtz).
- Un salto condicional se traduce como beqz sobre un j, porque beqz solo alcanza ±4 KB y j alcanza ±1 MB.
- Las constantes decimales se declaran en la sección .data como .double.

ecall	Uso
1	Imprimir entero
11	Imprimir carácter (las cadenas se decodifican de UTF-8 para mostrar tildes y ñ)

ecall	Uso
8	Leer una línea de la entrada (se consume carácter por carácter)
9	Reservar memoria (sbrk)
10	Terminar el programa

7. Ejecución desde la IDE

Menú Compilar

Opción	Acción
Analizar	Análisis léxico, sintáctico y semántico. Los errores se listan en la tabla.
Traducir a RISC-V	Genera main.asm, ejecutar.sh y ejecutar.bat en la carpeta del .pig.
Ejecutar assembler (.asm)	Ejecuta en RARS el .asm abierto, o el main.asm de la carpeta.
Traducir y ejecutar	Traduce y, si no hay errores, ejecuta.
Detener ejecución	Termina el proceso en ejecución.

Proceso de Ejecución

1. La IDE copia los scripts junto al .asm.
2. Lanza el script del sistema operativo: bash ejecutar.sh o cmd /c ejecutar.bat.
3. El script busca rars*.jar en este orden: la variable RARS_JAR, su propia carpeta y su lib/, y las carpetas superiores. Después ejecuta java -jar rars.jar nc me main.asm (nc: sin aviso de copyright; me: mensajes de RARS por la salida de error).
4. La salida del programa aparece en la pestaña Consola. El texto que se escribe en el campo inferior se envía como entrada al programa.